

Dokumen Pemilihan Tools, Library, dan Analisis Skalabilitas

SmartStock Pro - PT Maju Bersama Digital

1. Analisis Pemilihan Tools, Library, dan Framework

Untuk menjamin keandalan sistem "SmartStock Pro" tanpa membebani server dengan kompleksitas overhead yang tidak perlu, berikut adalah analisis dan justifikasi pemilihan teknologi yang digunakan:

A. Backend Framework: Node.js dengan Express.js

- **Alasan Pemilihan:**
 - **Asynchronous & Event-Driven:** Node.js menangani operasi I/O secara asinkronus, menjadikannya sangat cepat dalam memproses banyak koneksi HTTP simultan (seperti dari 5 gudang berbeda secara real-time) dengan penggunaan memori yang minimal.
 - **Ekosistem Package:** Memiliki ekosistem pustaka (npm) yang sangat besar dan matang untuk penanganan keamanan, manajemen file, database, dan pemrosesan paralel.
 - **Satu Bahasa Pemrograman:** Memungkinkan tim developer menggunakan JavaScript baik di sisi frontend maupun backend, mempercepat pengembangan dan mempermudah pemeliharaan kode.

B. Database Engine: SQLite3

- **Alasan Pemilihan:**
 - **Zero-Configuration & Portabilitas:** Seluruh database disimpan dalam satu file tunggal di server, sangat mudah untuk dideploy, dipindahkan, dan dicadangkan.
 - **Performa Server-Side:** Mendukung penuh transaksi ACID (Atomicity, Consistency, Isolation, Durability) dan relasi asing (Foreign Keys). Untuk aplikasi manajemen stok dengan volume data di bawah 10 juta baris, SQLite terbukti memiliki kecepatan pembacaan yang lebih tinggi daripada database Client-Server tradisional seperti MySQL karena mengeliminasi latensi soket jaringan.
 - **Transparansi SQL:** Membantu tim pengembang menulis query SQL relasional murni untuk melakukan penghitungan agregat stok, join produk, supplier, dan kategori secara langsung.

C. Library Pihak Ketiga (Third-Party Libraries)

Berikut adalah daftar library pihak ketiga yang digunakan dalam SmartStock Pro:

Library	Versi	Lisensi	Fungsi & Kegunaan dalam Sistem
Express	^4.19.2	MIT	Core routing engine untuk menangani request API HTTP, pengelolaan aset statis, dan server middleware.
BcryptJS	^2.4.3	MIT	Implementasi password hashing standar industri. Versi murni JavaScript menjamin kemudahan instalasi di berbagai platform (macOS, Linux, Windows) tanpa memerlukan compiler compiler native C/C++ pada server.
Express Session	^1.18.0	MIT	Mengelola session pengguna secara aman, memvalidasi timeout otomatis, dan menyediakan fondasi penyimpanan state otorisasi.
SQLite3	^5.1.7	BSD-3-Clause	Driver penghubung Node.js dengan engine database SQLite. Mendukung query asinkronus dengan prepared statements untuk proteksi SQL Injection.

2. Analisis Skalabilitas Sistem

PT Maju Bersama Digital memproyeksikan peningkatan jumlah transaksi seiring ekspansi bisnisnya. Untuk itu, arsitektur "SmartStock Pro" dirancang untuk menangani pertumbuhan data dan pengguna tanpa mengalami degradasi performa yang signifikan melalui strategi berikut:

A. Skalabilitas Vertikal (Scale-Up)

1. Database Indexing:

- Menambahkan indeks non-clustered pada kolom pencarian utama seperti `products.sku`, `products.name`, `warehouses.code`, dan `stock_transactions.transaction_code`.
- Mengindeks kolom `stock_batches.product_id` dan `stock_batches.warehouse_id` untuk mempercepat query pencarian batch aktif dalam perhitungan FIFO/LIFO.

2. Koneksi Database Pool:

- Menjaga koneksi database tetap terbuka (persistent connection) dan mengaktifkan fitur WAL (Write-Ahead Logging) di SQLite. Mode WAL memungkinkan operasi pembacaan (Read) berjalan secara konkuren

tanpa terhambat oleh proses penulisan data (Write) yang sedang berlangsung.

B. Skalabilitas Horizontal (Scale-Out)

Jika limitasi komputasi server tunggal tercapai, sistem dapat ditingkatkan ke arsitektur multi-server dengan langkah:

1. Migrasi Database ke PostgreSQL:

- SQLite3 diganti dengan PostgreSQL klaster (Primary-Replica). Struktur query SQL murni yang kita gunakan dirancang portabel sehingga migrasi database hanya membutuhkan penyesuaian minimal pada lapisan konfigurasi koneksi (`db.js`).

2. Session Sharing via Redis Store:

- Mengubah penyimpanan session default memori Express ke klaster **Redis**. Dengan demikian, status login pengguna dapat diakses oleh server aplikasi manapun di belakang Load Balancer.

3. Stateless App Servers:

- Seluruh server aplikasi Node.js dibuat bersifat *stateless* (tidak menyimpan data statis di lokal cakram). Semua file laporan dan aset upload dialihkan ke Object Storage (seperti AWS S3 atau MinIO).

C. Penanganan Pemrosesan Paralel

- **Worker Pools:** Menggunakan `Worker Threads` bawaan Node.js untuk mendistribusikan tugas komputasi berat (import CSV masif, generate laporan PDF besar) ke thread CPU yang terpisah. Hal ini menjaga event loop utama Node.js tetap luang untuk merespons request API pengguna dengan latensi di bawah 100ms.
- **Asynchronous Task Queue:** Transaksi transfer antar gudang diproses secara paralel menggunakan penanganan antrian internal untuk memastikan konsistensi pembaruan stok tanpa mengakibatkan kebuntuan (*deadlock*) database.